

Language Models

n-grams and word2vec

Schedule: Foundations

Sep 10	Tree-based + Neural Nets	Jonas
Sep 17	Language Models	Stephanie
Sep 24	Transformers	Stephanie
Oct 1	Generative AI	Stephanie
Oct 8	LLMs	Stephanie

Large Language Models

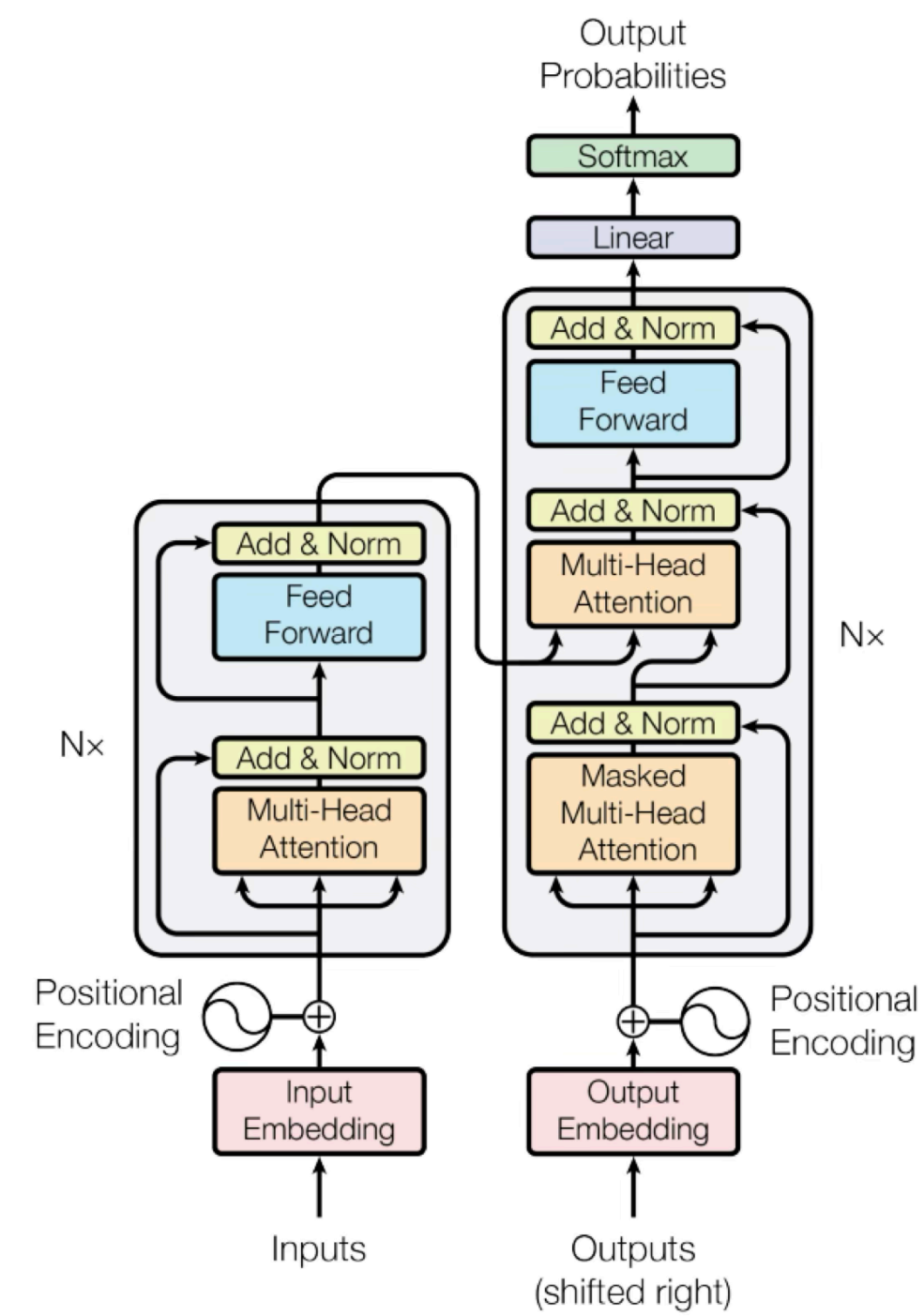


Figure 1: The Transformer - model architecture.

Large Language Models

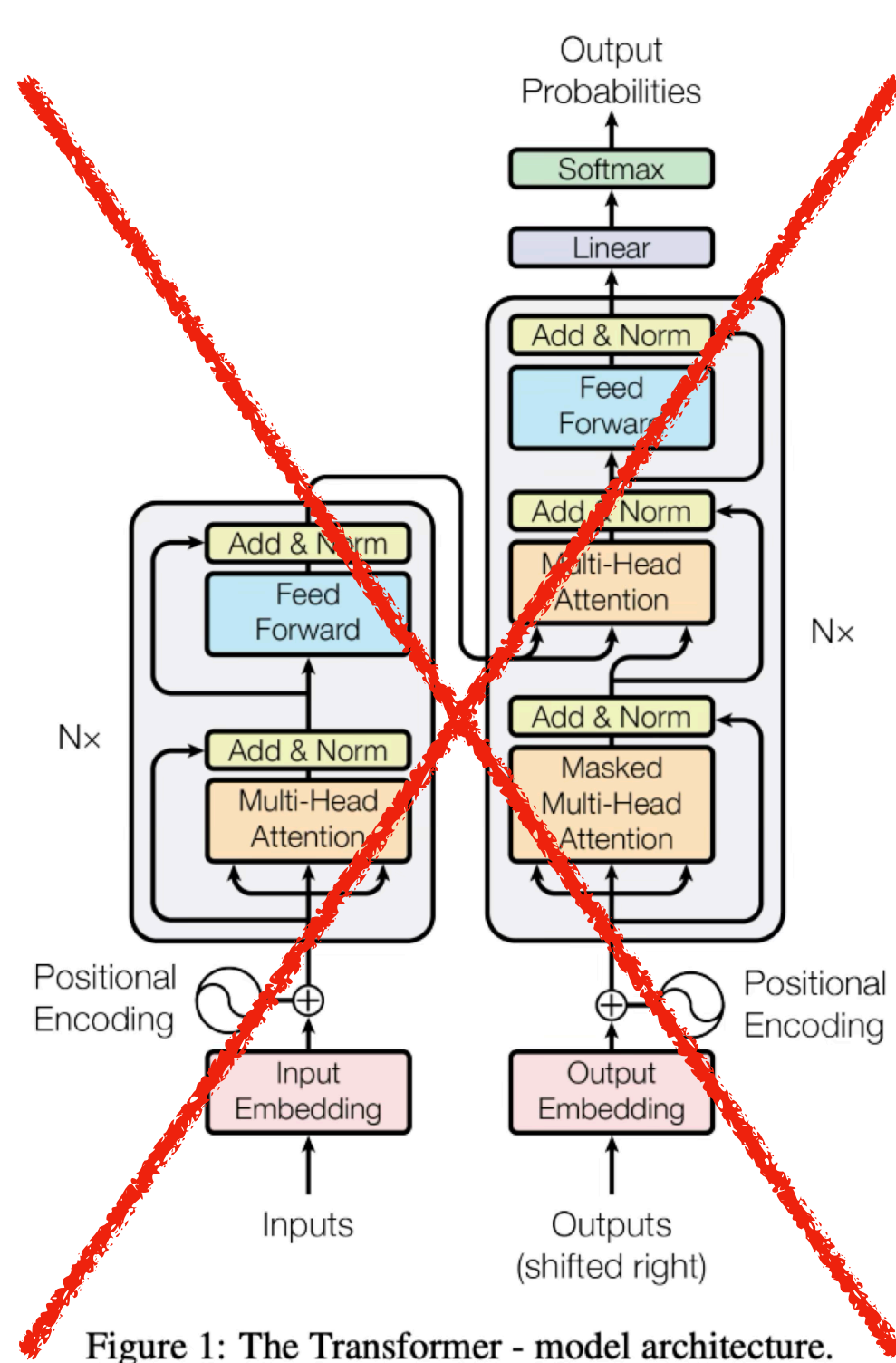
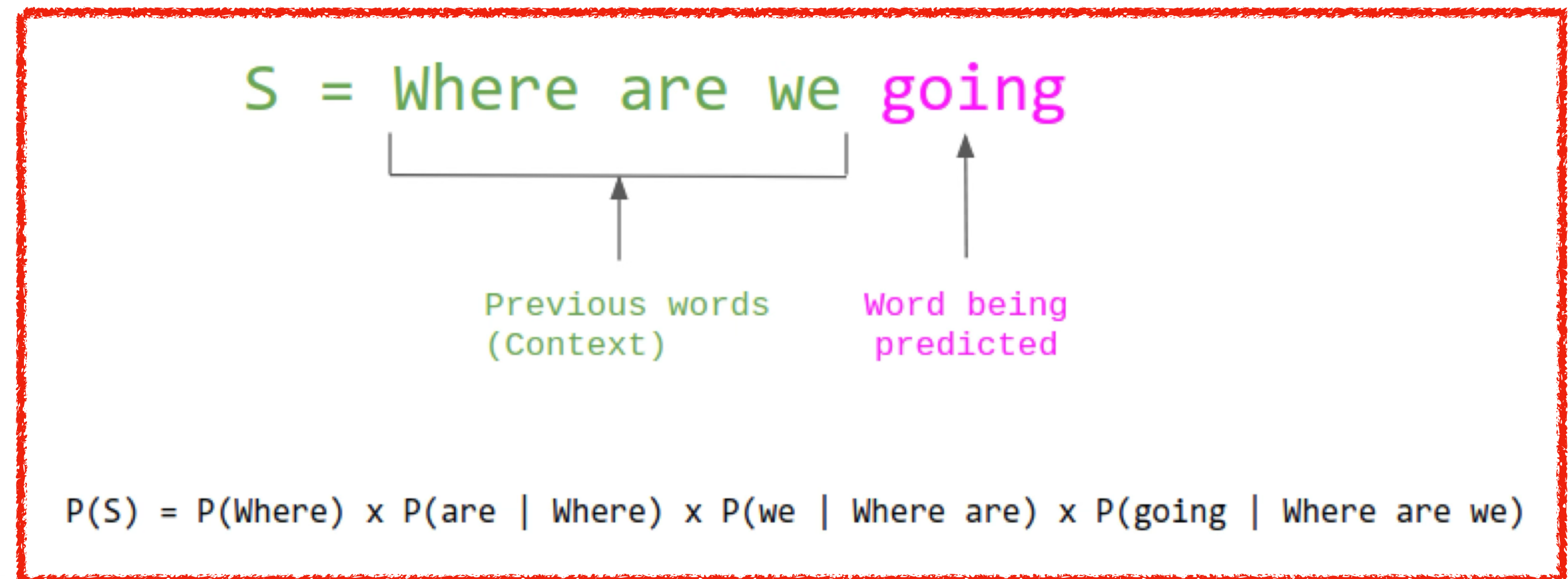


Figure 1: The Transformer - model architecture.



Agenda

- Language models
 - Definition
 - Formalization
 - n-grams
 - Evaluation
- Vector Semantics
 - Word Similarities
 - Co-occurrence matrices
 - TF-IDF weights
- word2vec
 - Algorithm
 - Evaluation
 - Visualization

Agenda

This lecture is based on

“Speech and Language Processing”, 2025. Jurafsky & Martin

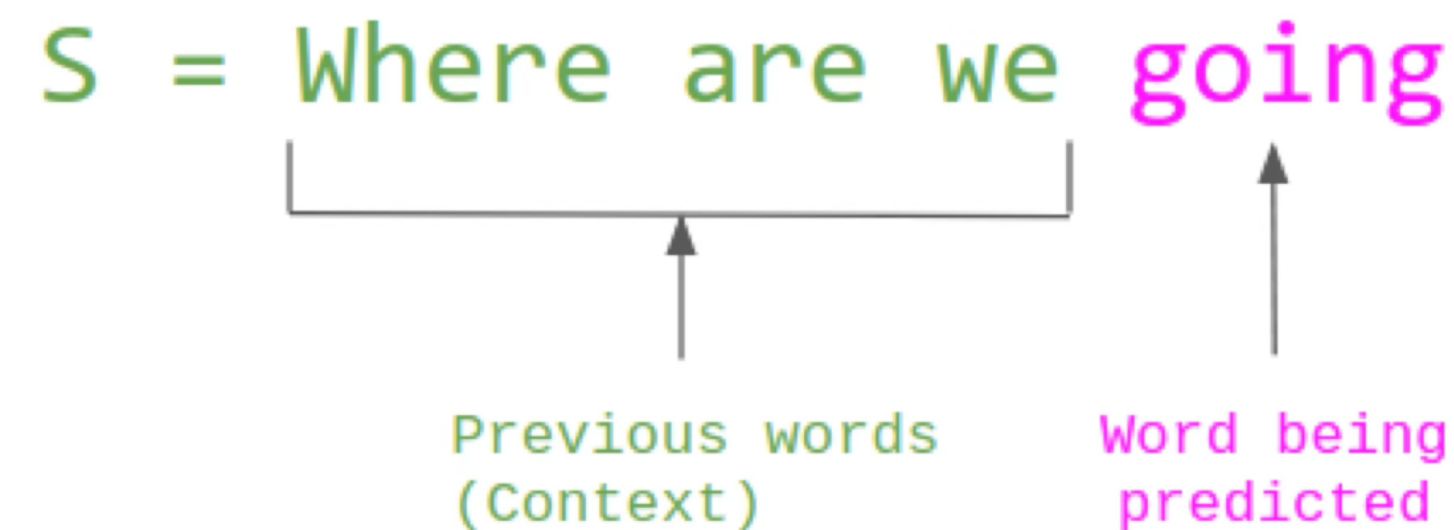
Additional resources:

Stanford CS224N: NLP with Deep Learning

Language Models

What is a language model?

“A language model assigns a probability to each possible next word or gives a probability distribution over possible next words”



$$P(S) = P(\text{Where}) \times P(\text{are} \mid \text{Where}) \times P(\text{we} \mid \text{Where are}) \times P(\text{going} \mid \text{Where are we})$$

What is a language model?

Formalization

Notation

$w_1, \dots, w_n$, simplified $w_{1:n}$, the sequence of n words

$P(X_i = w_i)$, simplified $P(w_i)$, the probability of a random variable X_i taking on the value (word) w_i

$P(w_k | w_{1:k-1})$, the probability of a word given a sequence of previous words

The joint probability over a sequence of words, e.g., a sentence $P(w_1, \dots, w_n)$ where we apply the chain rule:

$$\begin{aligned} P(w_1, \dots, w_n) &= P(w_1)P(w_2 | w_1)P(w_3 | w_{1:2}) \dots (w_n | w_{1:n-1}) \\ &= \prod_{k=1}^n P(w_k | w_{1:k-1}) \end{aligned}$$

Why is this relevant?

- spell/grammar correction

knowing the probabilities of words or sentences helps us understand that “Where are we going from here?” is more likely than “Where are we **gone** from here?”

- augmentative and alternative communication (AAC)

people unable to speak or sign can use AAC to communicate with gaze or brain signals

- large language models (LLMs)

LLMs are trained on next-token-prediction and generate sequences of text based on probabilities over their vocabulary

n-gram language models

- the simplest kind of language model: the n-gram language model
- an n-gram is a sequence of n words
- an n-gram language model is a probabilistic model that estimates the probability P of a word w given the n-1 previous words, i.e., the history h

$$P(w = \text{cold} \mid h = \text{The water that day was exceptionally})$$

n-gram language models

$$P(w = \text{cold} \mid h = \text{The water that day was exceptionally})$$

How do we estimate the probability P ?

- Naive approach: relative frequency count
Count the number of times “*Where are we*” is occurring in a large corpus and how often it is followed by the word “*going*”
 Problem: not a very flexible method, slight changes will not be considered, longer sentences are less likely to occur often enough
- n-gram approach: approximate the history with n-grams
for instance for a bigram model ($n = 2$), we make the assumption that

$$P(w_n \mid w_{1:n-1}) \approx P(w_n \mid w_{n-1})$$

$$P(w = \text{cold} \mid \text{exceptionally})$$

n-gram language models

Maximum Likelihood Estimation

How do we estimate the bigram probability $P(w_n | w_{n-1})$?

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)}$$

By counting all occurrences of the bigram $C(w_{n-1}w_n)$ in a corpus and normalize it by the sum of all bigrams that share the same first word w_{n-1} . For the case of bigrams, this can be simplified to

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

n-gram language models

- Let's create a mini-corpus containing the following sentences:
<s> I was here <\s>
<s> Yesterday I was here <\s>
<s> Were you here last night <\s>
- <s> and <\s> are beginning and end of sentence tokens to indicate a separation between sentences
- Next, we divide this corpus into bigrams:
<s> I ; I was; was here; here <\s>; <s> Yesterday; Yesterday I; <s> Were;
Were you; you here; here last; last night; night <\s>

n-gram language models

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

Corpus:

<s> I was here <\s>

<s> Yesterday I was here <\s>

<s> Were you here last night <\s>

bigram counts: <s> I (1) ; I was (2); was here (2); here <\s> (2); <s> Yesterday (1); Yesterday I (1); <s> Were (1);
Were you (1); you here (1); here last (1); last night (1); night <\s> (1)

unigram counts: <s> (3); I (2); was (2); here (3); <\s> (3); Yesterday (1); Were (1); you (1); last (1); night (1)

$P(I | <s>) =$

$P(\text{you} | \text{Were}) =$

$P(<\ s> | \text{here}) =$

$P(\text{night} | \text{here}) =$

Dealing with scale

- language model probabilities are usually stored and computed in log space as **log probabilities**
- this prevents numerical underflow, i.e., the number gets so small/close to zero that it can't be represented anymore by a computer
- adding in log space is equivalent to multiplying in linear space

$$p_1 \times p_2 \times p_3 \times p_4 = \exp(\log p_1 + \log p_2 + \log p_3 + \log p_4)$$

- usually, the natural log (ln) is used

Train-Val-Test Split

A machine learning concept

- When we train probabilistic language model, it is important to split our dataset in order to evaluate the model
- Usually, we split our data in **train-validation-test**
- train: that is the part where we train the model on
- validation/development: within the training process we use this split to validate our model
- test: after everything is done, we test our model on the test split

Train-Val-Test Split

in the case of n-gram models

- When we train probabilistic language model, it is important to split our dataset in order to evaluate the model
- Usually, we split our data in **train-validation-test** for instance a 80-10-10 split or a 70-20-10 split
- train: that is the part where we train the model on here, we compute the counts of n-grams that we normalize into probabilities
- validation/development: within the training process we use this split to validate our model we test our model on the validation split to see whether it works on unseen data, we might update parameters and use this to test again, e.g., change 4-grams to 5-grams
- test: after everything is done, we test our model on the test split this gives us an unbiased estimation of how well our models generalizes, we always measure performance on the test set

Evaluating language models

Intrinsic and extrinsic evaluation

- Intrinsic evaluation: measures the quality of a model independent of any application → **perplexity**
- Extrinsic evaluation: measure how much a specific application, e.g., machine translation, improves → **F1-scores or application-specific metrics**

Evaluating language models

Intrinsic and extrinsic evaluation

- Intrinsic evaluation: measures the quality of a model independent of any application → **perplexity**
- Extrinsic evaluation: measure how much a specific application, e.g., machine translation, improves → **F1-scores or application-specific metrics**

Perplexity

- A perfect language model correctly guesses each next word on a test set with a probability of 1 and assigns a probability of 0 to all other words.
- A language model that assigns higher probability to the words in a test set is considered “better” than the one assigning lower probabilities
- Raw probabilities over an entire test set depend on the the size (number of words) of the corpus, so we need a token-based metric and normalize by the number of words
- **Perplexity** is the inverse probability of the test set normalized by the number of words

$$\begin{aligned}\text{perplexity}(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}}\end{aligned}$$

Perplexity

- **Perplexity** is the inverse probability of the test set normalized by the number of words. For $W = w_1 \dots w_n$:

$$\text{perplexity}(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}}$$

$$= \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1 \dots w_{i-1})}}$$

- Because we look at inverse probabilities, the higher the probability of the word sequence, the lower the perplexity

Perplexity

- Because we look at inverse probabilities, the higher the probability of the word sequence, the lower the perplexity

The lower the perplexity of a model on the data, the better the model

- Think of perplexity as surprisal - we don't want the model to be too surprised by the next word
- The perplexity of a bigram model can be computed as

$$\text{perplexity}(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

Perplexity

- Perplexity can be used to compare the performance of different language models on the same text
- For this, the vocabulary has to be identical and the language model must be trained without knowledge of the test data
- Those are values of perplexity over a 1.5 million test set

	Unigram	Bigram	Trigram
Perplexity	962	170	109

- A trigram model has a lower perplexity, i.e., is less surprised, than a unigram model as it has seen more combinations of words during training

! Perplexity !

Rather !! evaluation !! in general

- Improvements in intrinsic evaluation does not guarantee improvements in extrinsic evaluation
- One should consider other factors when choosing a model
- A model with a lower perplexity score
 - might have been trained on more similar data
 - might not be suitable for your task
 - might be much bigger in parameters
- **It is always good to not rely on a single number!**

Summary

- n-gram models are transparent, easy to compute
- words are only represented as an index in a vocabulary
- we don't learn anything about word similarity between words in meaning like cat/mouse or syntax mouse/mice, they are all encoded in a similar way
- scales exponentially with the vocabulary
- we don't learn anything about what type of word it is (noun, verb, adjective) or what sentiment it carries

Vector Semantics

Word Embeddings

- Learning representations of the meaning of words, i.e., embeddings, directly from their distributions in text
- **Distributional hypothesis:** Words that occur in *similar contexts* tend to have *similar meanings*.
- With word embeddings, we can learn useful representation of input text in a self-supervised way (representation learning) instead of creating representations by hand (feature engineering).

Word similarity

Loose linguistic definitions

Synonymy: if one word has a sense whose meaning is (nearly) identical to a sense of another word, the two senses of those two words are synonyms.

big/huge, amazing/incredible, drink/beverage, car/automobile

Word Similarity: if two words overlap or resemble in meaning. Knowing the similarity between words helps us to compute the similarity between sentences/texts which is important when solving question answering, summarization etc.

(“is similar to”) cat/dog, king/queen, reading/writing

Word Relatedness: words that cover a particular semantic domain, antonyms (opposites), hyponym/hypernyms (superordinates/subordinates), meronyms (part-whole relation)

coffee/cup, fast/slow, fruit ↔ cherry, wheel/car

Word similarity

How to measure word similarity

One way to measure word similarity is by human judgement (at a scale 0-10), later we will see a way to do this with word embeddings

vanish	disappear	9.8
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

Vector Semantics

In vector semantics, we define a word by its **distribution** in language use, meaning its neighboring words (context) or grammatical environments.

Two words that occur in similar distributions/contexts, i.e., whose neighboring words are similar, have similar meanings.

Words are represented as **a point in a multidimensional vector space** that is derived from the distribution of word neighbors.

Those vectors are called embeddings or **word embeddings**.

Co-occurrence matrix

Term-document matrix

Word embeddings are generally based on a co-occurrence matrix, a way of representing how often words co-occur

In a **term-document matrix**, each row represents a word in the vocabulary and each column represents a document from some collection of documents.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

Co-occurrence matrix

Term-document matrix

Word embeddings are generally based on a co-occurrence matrix, a way of representing how often words co-occur

In a **term-document matrix**, each row represents a word in the vocabulary and each column represents a document from some collection of documents.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

not very informative →

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

Co-occurrence matrix

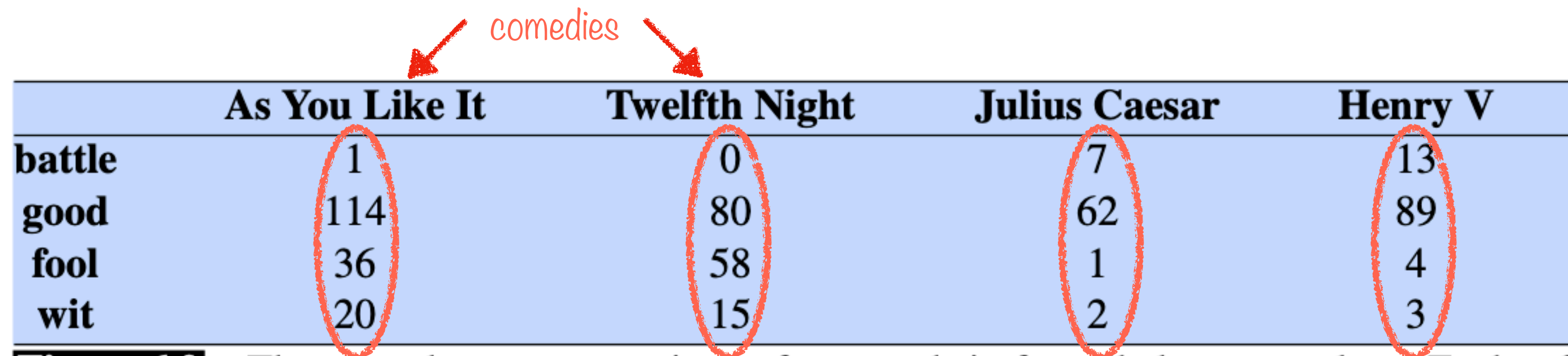
Term-document matrix

The term-document matrix was first defined in **information retrieval** where a **document** is represented as a count vector, a **column vector** below. In information retrieval, one task is to find similar documents. Under the assumption that similar documents tend to have similar words, we can assume that similar documents have similar count vectors.

A real term-document matrix would have many more rows and columns:

$|V|$ rows depending on the number of words in the vocabulary V

D columns, one for each document in the collection



	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

Co-occurrence matrix

Term-document matrix

We can also use term-document matrices to represent the meaning of words by associating each **word** with a vector, here a **row vector**.

The same principle applies as before: similar words have similar vectors because they tend to occur in similar documents

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

Co-occurrence matrix

Word-word matrix

An alternative to the term-document matrix is the word-word matrix where both columns and rows are labeled by words. The matrix is thus of dimensions $|V| \times |V|$ (the length of the chosen vocabulary). Each cell represents the number of times the row (**target**) word co-occurs with the column (**context**) word in a training corpus. The context could be defined as the entire document but usually it is defined as a window around the words, e.g., 4 words to the left and 4 words to the right.

The **water that day was** **exceptionally** **warm so she took** the opportunity to

	day	she	so	that	the	took	warm	was
exceptionally	1	1	1	1	1	1	1	1

Co-occurrence matrix

Word-word matrix

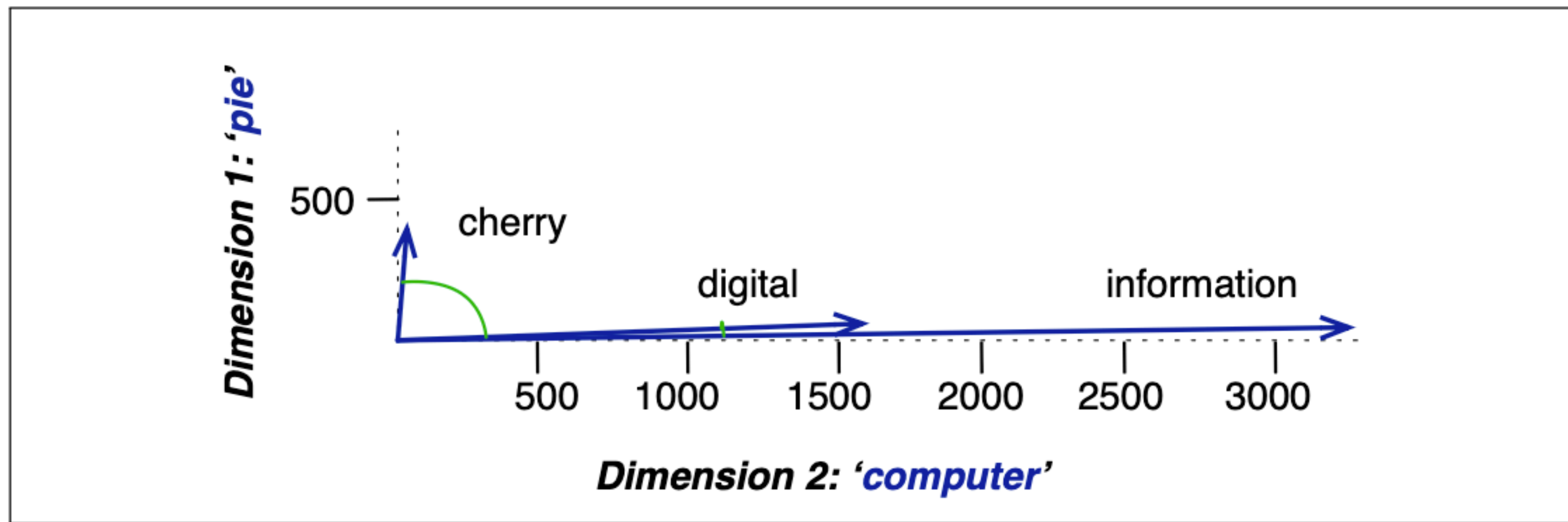
	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Figure 6.6 Co-occurrence vectors for four words in the Wikipedia corpus, showing six of the dimensions (hand-picked for pedagogical purposes). The vector for *digital* is outlined in red. Note that a real vector would have vastly more dimensions and thus be much sparser.

Cosine for measuring similarity

A measure of similarity requires two word vectors of the same dimension as input and a similarity score as output.

The most common similarity metric is the cosine of the angle between vectors.



Be aware that we usually do not know what the axes represent when visualizing word embeddings

Cosine for measuring similarity

$$\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \|\mathbf{w}\|} = \frac{\sum_{i=1}^d v_i w_i}{\sqrt{\sum_{i=1}^d v_i^2} \sqrt{\sum_{i=1}^d w_i^2}}$$

where $\mathbf{v}, \mathbf{w} \in \mathbb{R}^d$ and $\mathbf{v} \cdot \mathbf{w}$ represents the dot product

$$\text{dot product}(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \mathbf{v}^\top \mathbf{w} = \sum_{i=1}^d v_i w_i = v_1 w_1 + v_2 w_2 + \dots v_d w_d$$

and $\|\mathbf{v}\|, \|\mathbf{w}\|$ the length of the vectors, i.e., the dot product with itself

$$\|\mathbf{v}\| = \sqrt{\mathbf{v}^\top \mathbf{v}} = \sqrt{\sum_{i=1}^d v_i^2}$$

Back to co-occurrence matrices

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

- The co-occurrence matrices we've seen, contain a co-occurrence count either of words with documents or words with other words.
- This raw frequency count is not ideal to measure associations/relations between words:
 - very frequent words do not carry a lot of information (like "good" in the example above)
 - but we are still interested in frequent words that occur nearby a target word

TF-IDF Weighing terms in the vector

Term frequency - Inverse document frequency

- The tf-idf weighting tries to balance this problem by using two different counts
- The first is the **term frequency**:
the frequency of the word t in the document d : $\text{tf}_{t,d} = \text{count}(t, d)$
- Instead of using the raw count, we use $\log_{10}$. The intuition behind this is, that a word that appears 100 times in a document is not necessarily 100 times more relevant than a word that occurs once

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d) & \text{if } \text{count}(t, d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

TF-IDF Weighing terms in the vector

Term frequency - Inverse document frequency

- The second term is **inverse document frequency** which gives a higher weight to words that occur only in a few documents. Those words are useful to discriminate between documents.
- Document frequency df_t of a word t is the number of documents it occurs
- The inverse document frequency is the number of documents divided by df and as before we use the $\log_{10}$

$$idf_t = \log_{10} \frac{N}{df_t}$$

TF-IDF Weighing terms in the vector

Term frequency - Inverse document frequency

- Bringing both terms together yields (for $\text{count}(t, d) > 0$)

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

$$= (1 + \log_{10} \text{count}(t, d)) \cdot \log_{10} \frac{N}{\text{df}_t}$$

TF-IDF Weighing terms in the vector

Term frequency - Inverse document frequency

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.246	0	0.454	0.520
good	0	0	0	0
fool	0.030	0.033	0.0012	0.0019
wit	0.085	0.081	0.048	0.054

Figure 6.9 A portion of the tf-idf weighted term-document matrix for four words in Shakespeare plays, showing a selection of 4 plays, using counts from Fig. 6.2. For example the 0.085 value for *wit* in *As You Like It* is the product of $tf = 1 + \log_{10}(20) = 2.301$ and $idf = .037$. Note that the idf weighting has eliminated the importance of the ubiquitous word *good* and vastly reduced the impact of the almost-ubiquitous word *fool*.

Summary

- TF-IDF vectors are sparse long word vectors of length $|V|$
- they are based on co-occurrence matrices
- words that appear in similar contexts/documents are also similar in their embeddings
- we can compute similarity values based on the cosine similarity
- we now look into dense shorter word vectors: word2vec
- the dimensions are not directly interpretable
- they work better in NLP tasks and scale better

word2vec

word2vec

Motivation

- short dense vector representation
- number of dimensions ranging from 50 - 1000 instead of $|V|$ (>10.000)
- unlike tf-idf the dimensions don't have a clear interpretation
- dense vectors work better in NLP applications than sparse vectors:
 - the classifier has to learn fewer weights
 - smaller parameter space helps with generalization and prevents overfitting
- fast, efficient to train

word2vec

Intuition

- static embeddings, one fixed embedding for each word in the vocabulary
- instead of counting co-occurrences, a classifier is trained on a binary prediction task: “Is word w likely to appear next to word t ?”
- we don’t really care about the prediction but we learn embeddings while doing this
- we train this on running text in a self-supervised manner: no need for hand-labeling features (similar to tf-idf)

word2vec

In a nutshell

1. Treat the target word and a neighboring context word as positive examples
2. Randomly sample other words in the lexicon to get negative samples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights as embeddings

word2vec

Let's put this in numbers

Let's take the following sentence as our example:

... lemon, a [tablespoon of apricot jam, a] pinch ...
 c1 c2 w c3 c4

and “apricot” as our target word with a context window of 2

Our goal is to train a classifier such that, a given tuple (w, c) of a target word w paired with a potential context word c will return the probability that c is a real context word

$$P(+ | w, c)$$

word2vec

How to compute P ?

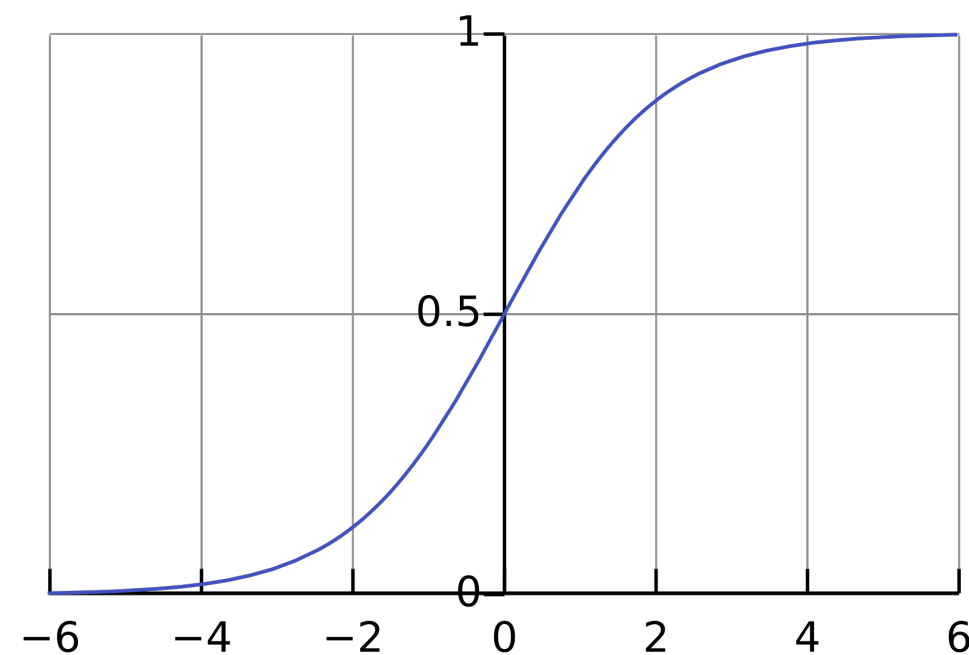
The intuition behind word2vec is that a word is more likely to occur near the target word if their embedding vectors are similar. Similar to the cosine similarity, we use the dot product as similarity measure

$$\textit{Similarity}(w, c) \approx \mathbf{c} \cdot \mathbf{w} \in [-\infty, \infty]$$

To turn this into a probability, we use the sigmoid function $\sigma(x)$

$$\sigma(x) = \frac{1}{1 + \exp(-x)} \in [0, 1]$$

$$P(+ | w, c) = \sigma(\mathbf{c}, \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{c} \cdot \mathbf{w})}$$



Sigmoid function

word2vec

How to compute P ?

$$P(+ | w, c) = \sigma(\mathbf{c}, \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{c} \cdot \mathbf{w})}$$

This is the probability for a single context word, but we need the probabilities over all context word in the vocabulary. For this, we assume that all context words are independent.

$$P(+ | w, c_{1:L}) = \prod_{i=1}^L \sigma(\mathbf{c}_i \cdot \mathbf{w})$$

$$\log P(+ | w, c_{1:L}) = \sum_{i=1}^L \log \sigma(\mathbf{c}_i \cdot \mathbf{w})$$

word2vec

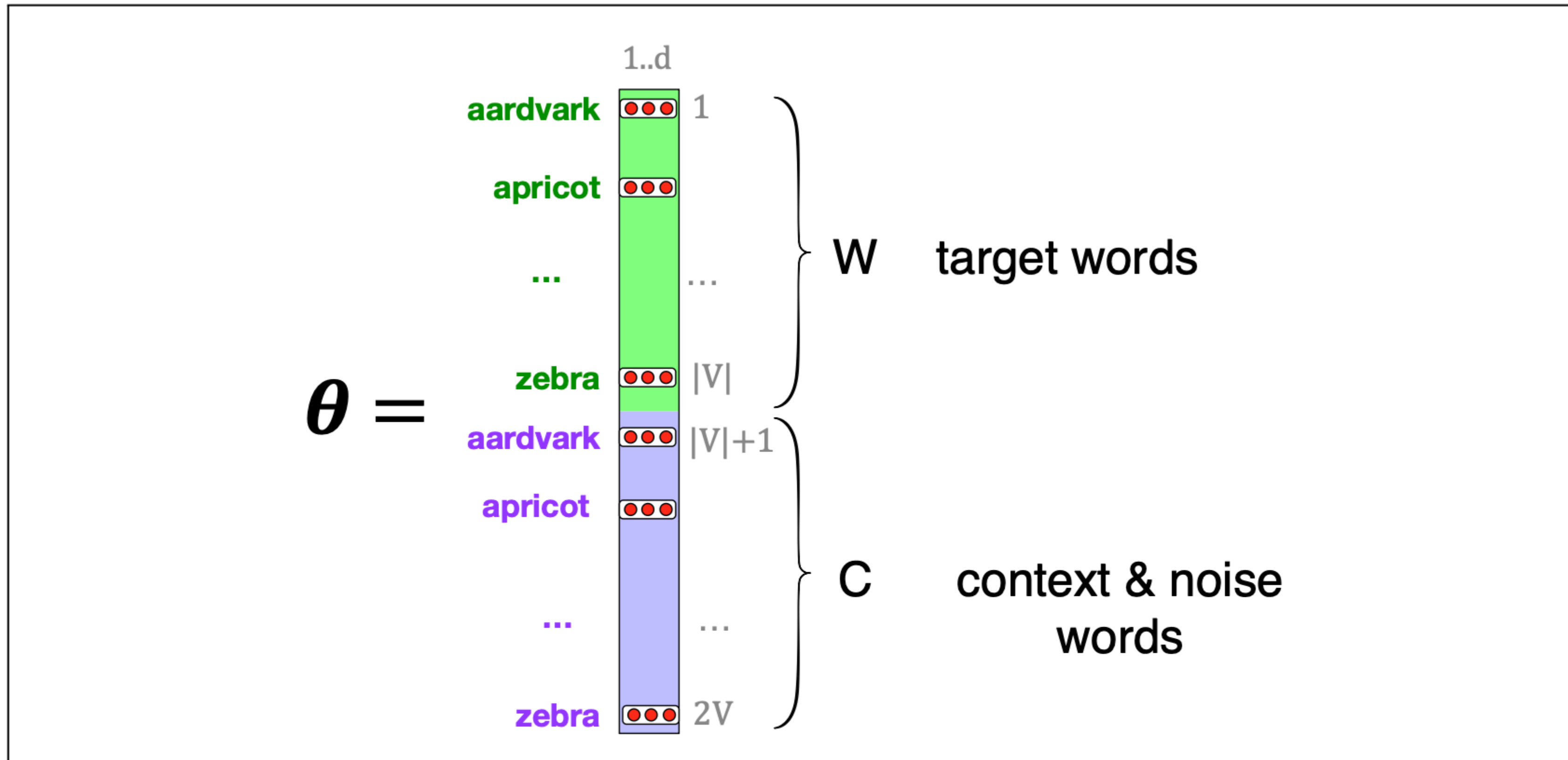


Figure 6.13 The embeddings learned by the skipgram model. The algorithm stores two embeddings for each word, the target embedding (sometimes called the input embedding) and the context embedding (sometimes called the output embedding). The parameter θ that the algorithm learns is thus a matrix of $2|V|$ vectors, each of dimension d , formed by concatenating two matrices, the target embeddings **W** and the context+noise embeddings **C**.

word2vec

Learning embeddings - Algorithm step by step

We start with a corpus of text, a vocabulary of size N .

Random embedding vectors are assigned for each of the N words in the vocabulary. During the optimization, embeddings are iteratively updated and thus shifted towards “meaningful” embeddings.

... lemon, a [tablespoon of apricot jam, a] pinch ...
 c1 c2 w c3 c4

positive examples +

 w c_{pos}

apricot tablespoon

apricot of

apricot jam

apricot a

negative examples -

$$w$$
 c_{neg} $\mathcal{W}$ C_{neg}

apricot aardvark apricot seven

apricot my apricot forever

apricot where apricot dear

apricot coaxial apricot if

word2vec

Learning embeddings - Algorithm step by step

Given the sets of positive and negative training instances, and an initial set of embeddings, the goal of the learning algorithm is to adjust those embeddings to

- **Maximize** the similarity of the (target word, context word) pairs (w, c_{pos}) drawn from the positive examples
- **Minimize** the similarity of the (w, c_{neg}) pairs from the negative examples

$$L = -\log \left[P(+ | w, c_{pos}) \prod_{i=1}^k P(- | w, c_{neg_i}) \right]$$
$$= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

for detailed math, check p. 121 of “Speech and Language Processing” by Jurafksy and Martin

word2vec

Learning embeddings - Algorithm step by step

Given the sets of positive and negative training instances, and an initial set of embeddings, the goal of the learning algorithm is to adjust those embeddings to

- **Maximize** the similarity of the (target word, context word) pairs (w, c_{pos}) drawn from the positive examples
- **Minimize** the similarity of the (w, c_{neg}) pairs from the negative examples

maximize probability of the positive examples
to be neighbors with the target word

$$L = -\log \left[P(+ | w, c_{pos}) \prod_{i=1}^k P(- | w, c_{neg_i}) \right]$$

maximize probability of the negative examples
NOT being neighbors with the target word

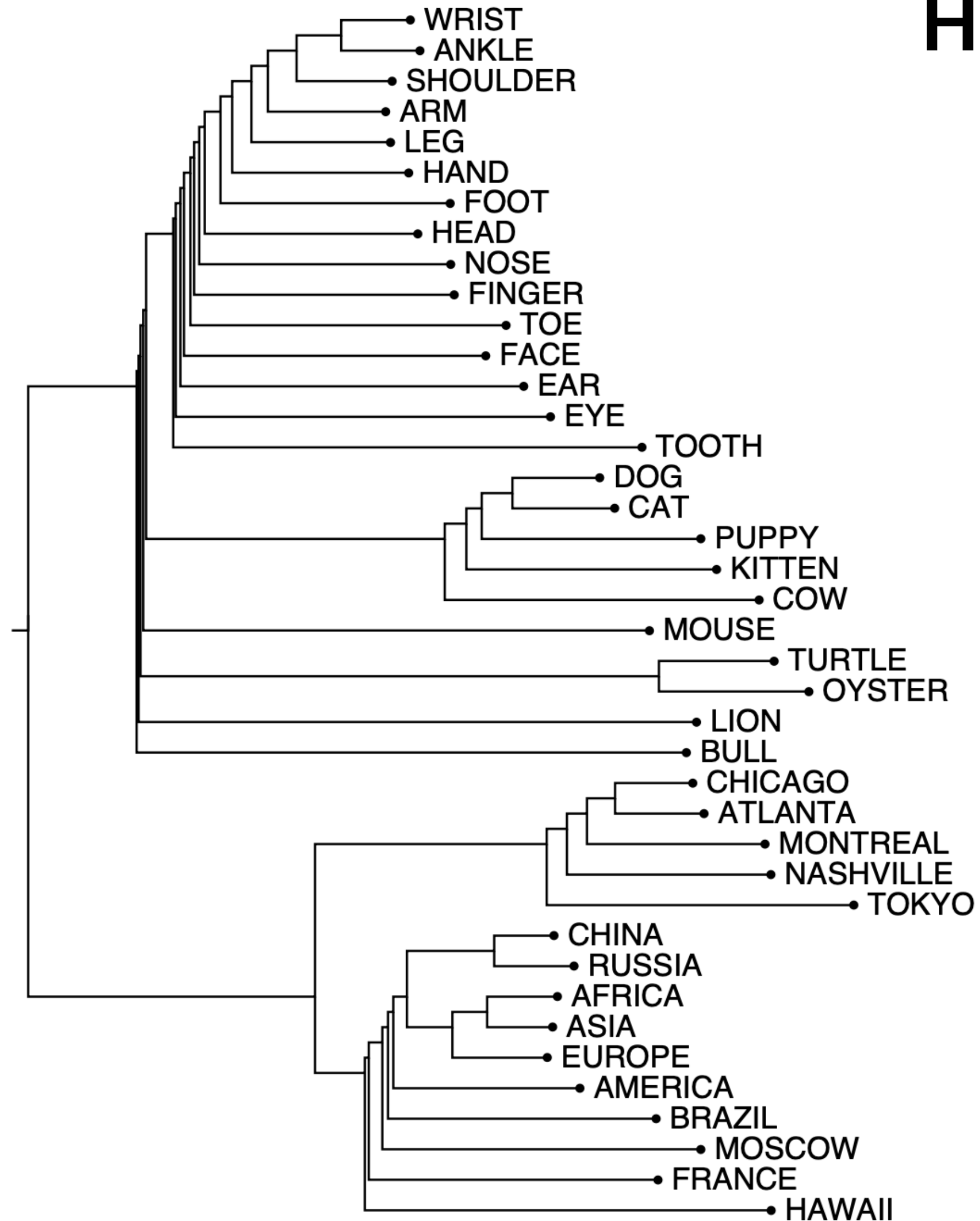
$$= \text{--} \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

L is being minimized

for detailed math, check p. 121 of "Speech and Language Processing" by Jurafksy and Martin

Visualizing embeddings

Hierarchical clustering



to be continued

Evaluating embeddings

- if possible use extrinsic evaluation on a task to see if performance is improved
- intrinsic evaluation:
 - compare word similarity of embeddings with word similarity of humans
 - the analogy task

Evaluating embeddings

Queen is to King as Women is to ?

Embeddings are able to capture relational meanings and can solve analogy problems like

a is to be as a is to what?*

apple is to tree as grape is to _____

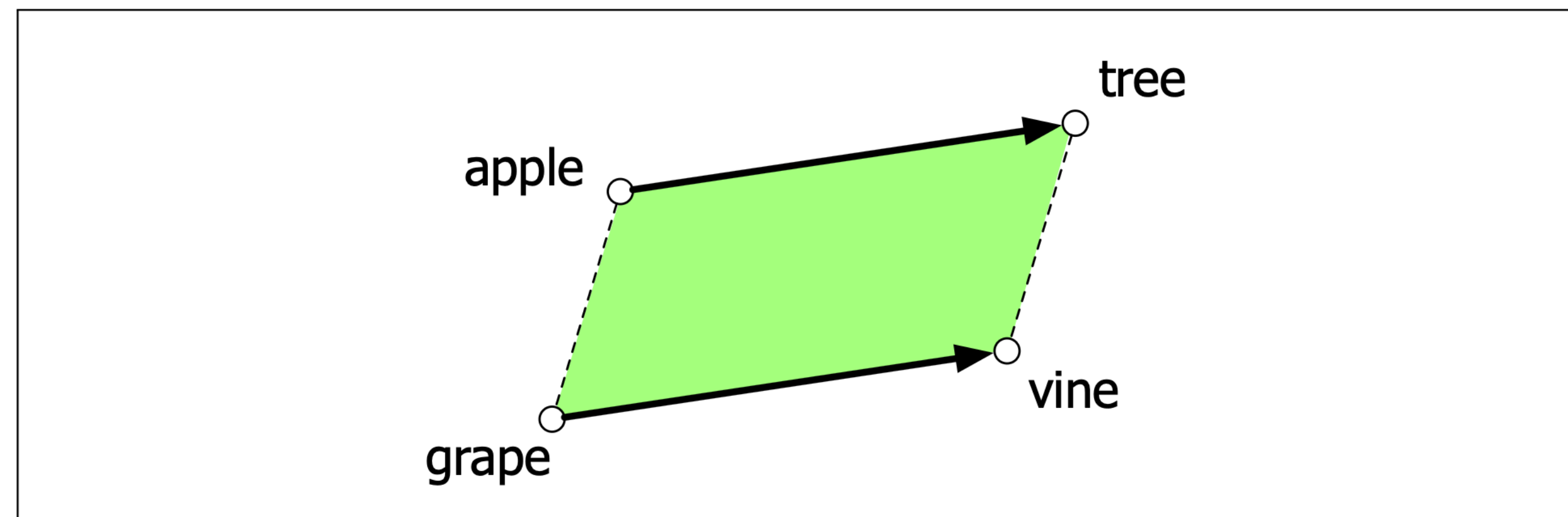


Figure 6.15 The parallelogram model for analogy problems (Rumelhart and Abrahamson, 1973): the location of $\vec{\text{vine}}$ can be found by subtracting $\vec{\text{apple}}$ from $\vec{\text{tree}}$ and adding $\vec{\text{grape}}$.

Evaluating embeddings

Queen is to King as Women is to ?

a is to b as a is to what?*

$$a : b :: a^* : b^*$$

$$\tilde{\mathbf{b}}^* = \arg \min_x \text{distance}(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{a}^*)$$

Some examples:

queen:king :: woman: ?

Paris:France :: Rome: ?

big:bigger :: small: ?

Bias and Embeddings

man:computer programmer :: woman: ?

Word embeddings are solely based on their training corpora, so they will also reflect and even amplify biases and opinions that are prominent in the data.

Bolukbasi et al., found that word2vec solves

man:computer programmer :: woman: homemaker

father:doctor :: mother: nurse

This can lead to allocation harm, where a system allocates resources (jobs or credit) unfairly to different groups. [Crawford (2017), Blodgett (2020)]

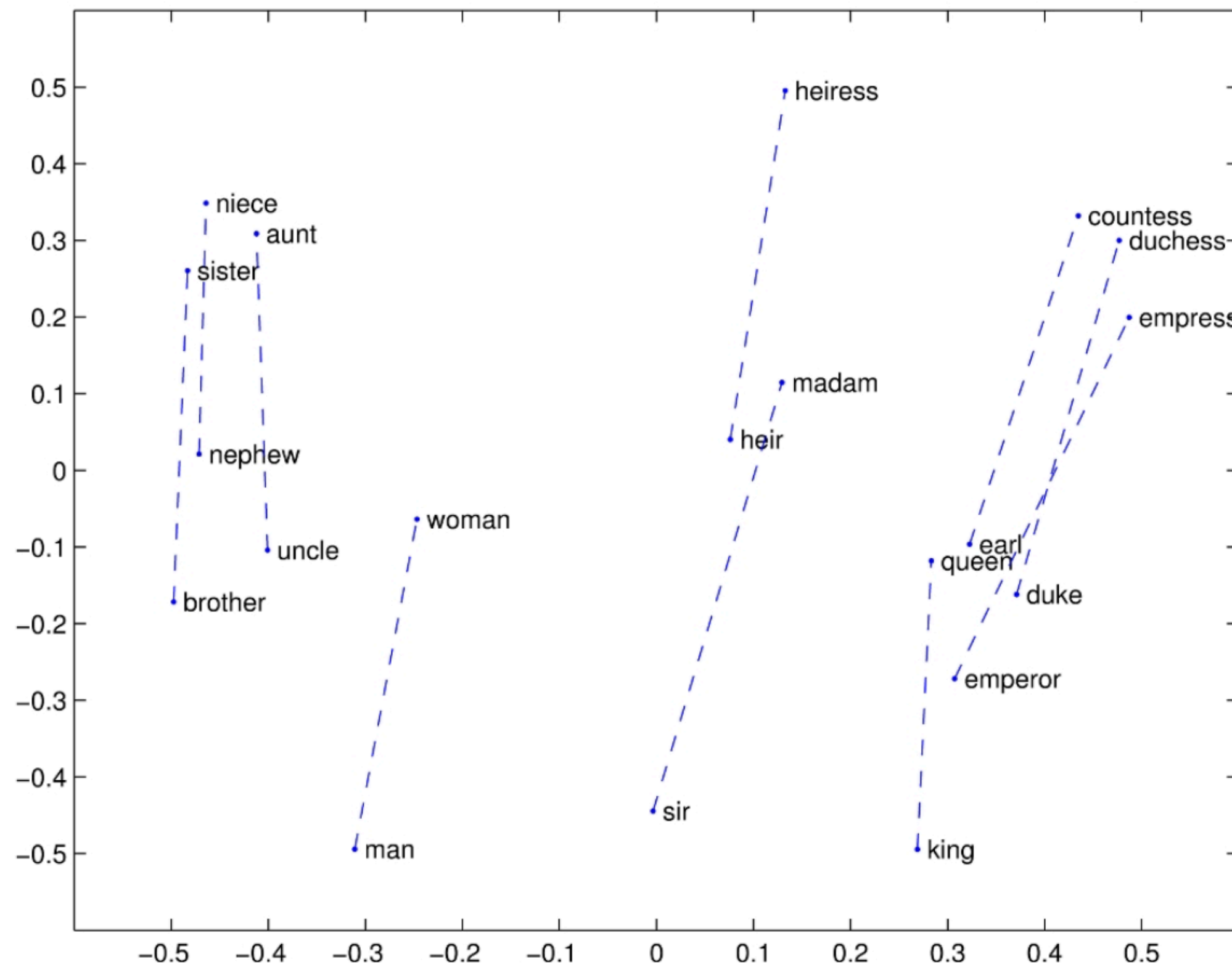
More on this in Lecture 7 ✨

Language (Technology) is Power: A Critical Survey of “Bias” in NLP (Blodgett et al., ACL 2020)

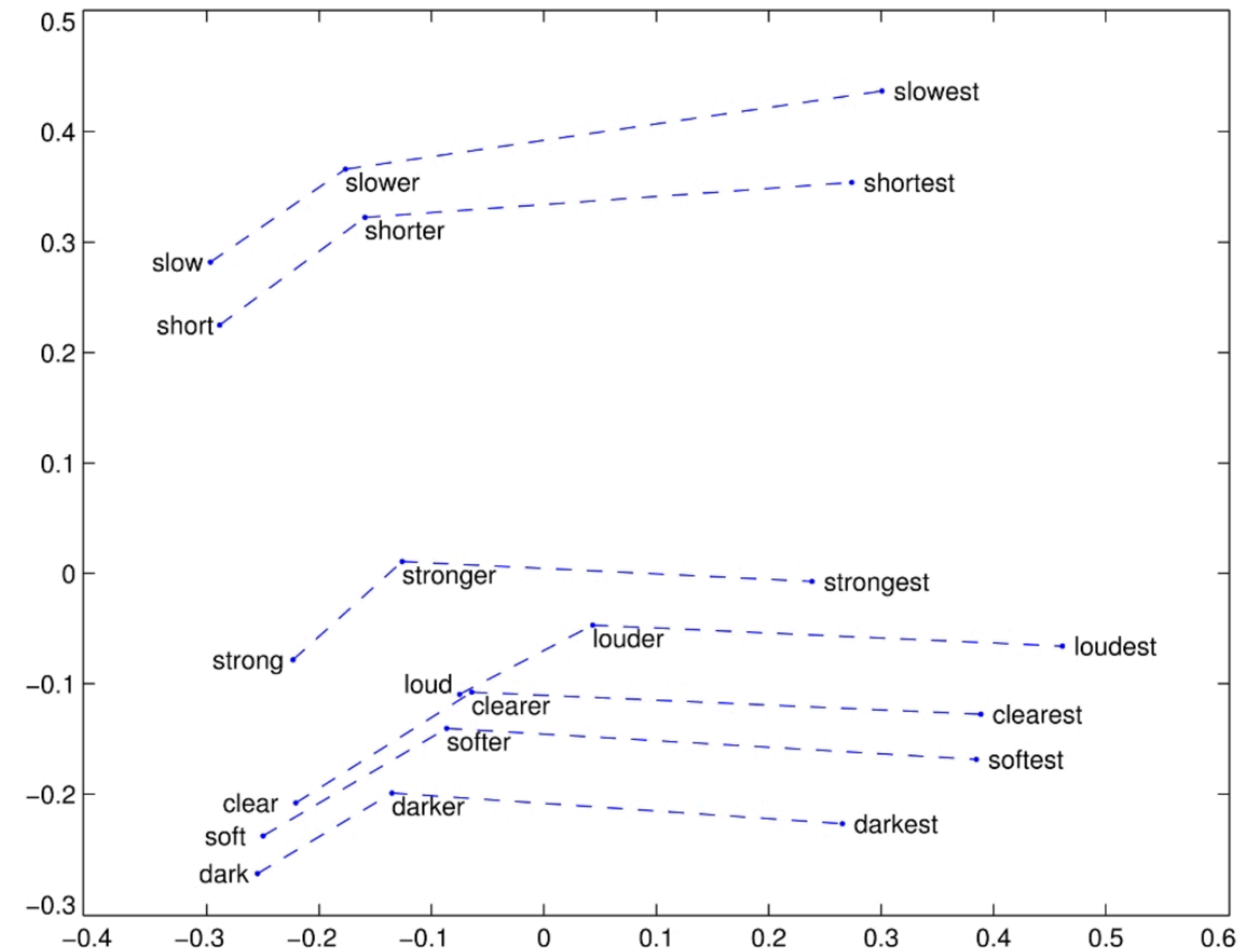
Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings (Bolukbasi et al, NeurIPS 2016)

Visualizing embeddings

Trajectories in 2D



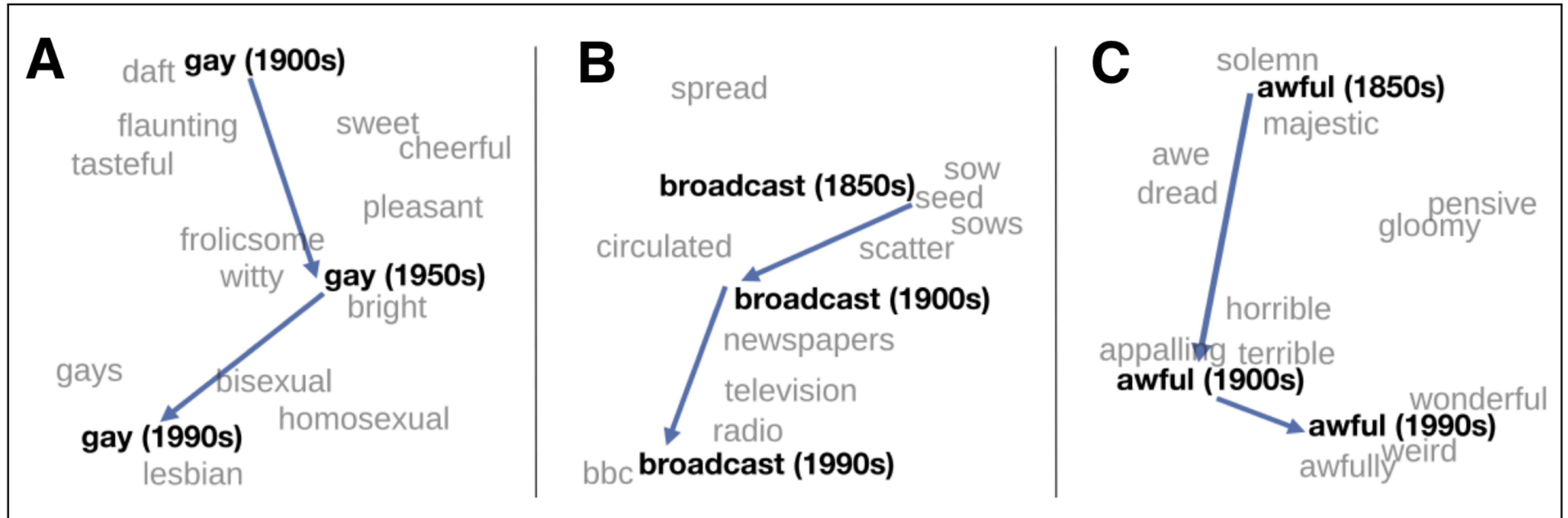
(a)



(b)

Visualizing embeddings

Trajectories in 2D



Conclusion & Outlook

- language models don't need to have billions of parameters
 - defining the next-word-probability is the crucial part
 - word embeddings can be sparse with many dimensions but work better as shorter and dense vectors
 - they can be used for visualization and vector arithmetic
-
- next week we will look into the Transformer architecture - the backbone of LLMs
 - we will focus on the attention mechanism